

StudentIDs:	293867	Score:
Class:	Data Warehouse Mini-Project	
Date:	16-06-2026	

This mini project assignment consists of 2 tasks. You should focus on providing complete solution, however if you are unable to solve a particular step, try to give at least a partial solution or provide justification for the reason for the lack of a solution.

MINI-PROJECT 3 – IMPLEMENTATION

The focus of this stage is on implementation tasks: implementation of the ETL process using SQL Server Integration Services (SSIS) and implementation of a multidimensional cube (OLAP Cube) using SQL Server Analysis Services (SSAS). You will utilise the artifacts prepared in the previous stage, where you have managed to design a detailed multidimensional model schema (star/snowflake) and an overall ETL process map.

The draft of the map serves as the blueprint for the ETL process. Now your task is to implement the process. Bear in mind that due to limited resources (mostly time) we focus on a simplified task of implementing an exemplar ETL solution, rather than building a complete, more general, ETL system. You have already specified all needed data movement, cleaning, and conformation tasks. Now it is time to act. As such, in Task 1 you need to implement the planned pipeline using SSIS (if you prefer to use a different ETL tool, or set of tools, please consult your selection with the teacher – but generally it is possible and encouraged).

TASK 1 – ETL PROCESS – DETAILS:

Before the start, please divide your dataset into historical load data (data that is initially loaded into your data mart) and current load data batches (at least, two data batches that can be later loaded incrementally). Please prepare appropriate implementation of the historical load (remember to create needed target data structures) and incremental loads (remember to automate the entire process, keep metadata of your runs). Obviously, both loads require proper handling of data cleaning and conforming tasks – as planned in your ETL map.

If you create a solution that can handle only historical load of data your final score for this part of the MP is highly limited, you can get a passing grade, but it is limited to 65%. Moreover, solutions which do not introduce two separate solutions (understood, as two completely different systems that are triggered separately) to handle historical and incremental loads are preferred.

ETL implementation – using a tool of your choice (suggested SSIS and MS SQL Server, remember about staging the data):

1. Divide your data into three batches (first batch can be, and should be, significantly larger)
2. Historical load of data – with the first batch
 - a. Define target data structures – required dimension tables and a fact table
 - b. Perform extraction of needed data from the source system. Please consider limiting the extraction to small portions of data (like 10000 rows), you'll find that this is a required approach for larger datasets (required for >85%).
 - c. Perform basic anomaly detection and resolution transformations. Define a basic set of data-quality screens and implement them (and further run against extracted data). Please note that a simple data-quality screen can check for null values and mark the rows as requiring attention. If you are using SSIS, please note that more advanced screens can be implemented using external scripts. In short, define an approach and clean the identified anomalies
 - d. Perform needed data transformation - creation of assumed multi-dimensional scheme - star, snowflake, or fact constellation:
 - i. Create dimension tables. Remember to integrate and standardize the needed information, to make names and values verbose, to introduce surrogate keys, to handle SCDs, please also include basic sanity checks against the completeness and correctness of data, e.g., row counts. Consider implementing automated tests (required for >85%).
 - ii. Create fact table. Remember to integrate and standardize the needed information. Please also include basic sanity checks against the completeness and correctness of data, e.g., row counts. Consider implementing automated tests (required for >85%).
 - e. Finally load the prepared data into target tables.
 - i. Define proper relations between dimensions and fact tables.
 - f. Use proper logging mechanisms and proper error handling (required for >85%). Remember to capture all erroneous data.
3. Incremental load of data – with the second and third batch
 - a. Identify required extraction approach and implement it – remember to utilise a mechanism to capture only new/modified data (required for >65%). Perform extraction of needed data from the source system

- b. Perform basic anomaly detection and resolution transformations. Define a basic set of data-quality screens and implement them (and further run against extracted data). Please note that a simple data-quality screen can check for null values and mark the rows as requiring attention. If you are using SSIS, please note that more advanced screens can be implemented using external scripts. Consider implementing automated quality tests (required for >50%). In short, define an approach and clean the identified anomalies.
- c. Perform needed data transformation - creation of assumed multi-dimensional scheme - star, snowflake, or fact constellation. Create dimension and fact tables. Remember Implement a mechanism to update only new/modified data (required for >65%)
- d. Finally load the prepared data into target tables.
- e. Use proper logging mechanisms and proper error handling (required for >85%). Remember to capture all erroneous data.

As the final solution, please upload your source files. In the final report provide a concise description of your implementation in accordance with the following points:

1.1 TARGET DATABASE CREATION SCRIPT

Please specify target database creation script:

Two SQL scripts create all required database objects. They are idempotent and can be re-run safely without dropping existing data.

01_staging_schema.sql - Creates database *BTC_Staging* with tables *STG_BLOCKS*, *STG_TRANSACTIONS*, *STG_TX_INPUTS*, *STG_TX_OUTPUTS*, *STG_MARKET_DAILY*, *STG_FEAR_GREED_RAW*, *STG_PIPELINE_LOG*, and helper views *V_TX_INPUT_TOTALS*, *V_TX_OUTPUT_TOTALS*, *V_TX_PRIMARY_SCRIPT*.

02_dw_schema.sql - Creates database *BTC_DW* with star-schema tables *DIM_DATE* (seeded from 2009-01-03 through today), *DIM_BLOCK*, *DIM_TX_TYPE*, *DIM_MARKET*, *FACT_TRANSACTION*, and error-sink *ERR_QUALITY_REJECTS*. All foreign key constraints and covering indexes are created in the same script.

1.2 ETL PROCESSES

Please specify stages of your implemented approach and provided detailed description of each stage, provide details for each target table. Please specify tools that you have used and highlight their purpose in the process.

Tools used

Python 3.10+ (btc_pipeline.py) - core ETL engine; orchestrates all six pipeline steps, implements the watermark logic, and executes all DQ gates. pyodbc - connects Python to SQL Server via ODBC Driver 17; used for all staging inserts and DW load SQL. requests - HTTP client for mempool.space and Alternative.me REST API calls, including retry-with-backoff logic. yfinance - Yahoo Finance wrapper; fetches full BTC-USD OHLCV history and incremental daily updates without an API key. PyYAML - reads config.yaml at startup to resolve server names, batch sizes, and log paths without hardcoding. schedule - optional recurring scheduler (--schedule flag); runs the pipeline on a configurable day interval. SQL Server 2019/2022 - hosts both BTC_Staging (staging layer) and BTC_DW (star schema) on the same instance so cross-database three-part-name references (BTC_Staging.dbo.) resolve correctly inside DW load SQL.*

Stage 1 - Blocks (Step 1/6) → STG_BLOCKS

Source: mempool.space /api/v1/blocks (public REST API, no key required). Watermark: last_block_height in STG_PIPELINE_LOG. On first run (NULL watermark) the pipeline fetches blocks_per_run (10) blocks from the current chain tip downward. On subsequent runs it fetches only blocks with height above the stored watermark. Each block record provides: block_hash, height, version, timestamp, tx_count, size, weight, difficulty, merkle_root, nonce, bits, total_fees_sat, and pool attribution (pool_name, pool_slug from the extras payload). Records are bulk-inserted into STG_BLOCKS; duplicate heights are skipped via WHERE NOT EXISTS. The watermark is updated to MAX(height) at the end of the step. Target table columns: block_hash VARCHAR(64) PK, height INT UNIQUE, version INT, block_timestamp DATETIME, tx_count INT, block_size INT, block_weight INT, difficulty FLOAT, merkle_root VARCHAR(64), nonce BIGINT, bits VARCHAR(10), total_fees_sat BIGINT, pool_name VARCHAR(100), pool_slug VARCHAR(100), is_active BIT, load_ts DATETIME2.

Stage 2 - Transactions, Inputs, Outputs (Step 2/6) → STG_TRANSACTIONS, STG_TX_INPUTS, STG_TX_OUTPUTS

Source: mempool.space /api/block/{hash}/txs/{start_index} (paginated, 25 TXs per page). For each new block, the pipeline fetches up to max_tx_per_block (100) transactions by paginating through the API. Each transaction response provides: txid, version, locktime, size, weight, fee, fee_rate, is_coinbase, has_witness, is_rbf, primary_script_type (derived via V_TX_PRIMARY_SCRIPT from I/O script types), plus all vin (inputs) and vout (outputs). STG_TRANSACTIONS stores one row per TX with aggregate fee and flag columns. STG_TX_INPUTS stores one row per input with prev_txid, prev_vout, value_sat, script_type, and has_witness. STG_TX_OUTPUTS stores one row per output with vout_index, value_sat, script_type, and address. All three tables use txid as the FK anchor; duplicates are skipped via WHERE NOT EXISTS on txid.

Stage 3 - Market OHLCV (Step 3/6) → STG_MARKET_DAILY

Source: Yahoo Finance via `yfinance` (BTC-USD ticker, daily OHLCV). No API key required; `yfinance` handles internal throttling. Watermark: `MAX(price_date)` in `STG_MARKET_DAILY`. First run fetches full history from 2014-09-17 (earliest Yahoo Finance BTC data) to today. Subsequent runs fetch only dates after the watermark. A 7-day simple moving average of the closing price is computed in SQL after insert. A 14-day realised volatility (annualised standard deviation of daily log-returns) is computed in Python using the `pandas` rolling `std` function before insert. The daily average price (`btc_price_avg_usd`) is computed as $(open + high + low + close) / 4$ and stored directly in the staging row. Target: `STG_MARKET_DAILY` with columns `price_date` DATE PK, `price_open`, `price_high`, `price_low`, `price_close`, `price_avg`, `volume`, `ma7`, `volatility_14d`, `fear_greed_score`, `fear_greed_label`, `btc_dominance_pct`, `market_cap_usd`.

Stage 4 - Fear & Greed Index (Step 4/6) → STG_FEAR_GREED_RAW then merged into STG_MARKET_DAILY

Source: `Alternative.me /api/fng/?limit=0` (returns full history of up to 3,000+ records, no key required). Watermark: `MAX(fg_date)` in `STG_FEAR_GREED_RAW`. First run fetches the entire history; subsequent runs fetch only records after the watermark. Raw records land in `STG_FEAR_GREED_RAW` (`fg_date` PK, `fg_score` TINYINT, `fg_label` VARCHAR, `fg_timestamp` BIGINT). After insert, an `UPDATE` joins `STG_FEAR_GREED_RAW` to `STG_MARKET_DAILY` on date equality to populate `fear_greed_score` and `fear_greed_label` in the market table. The two-table design keeps the raw sentiment history separate from the aggregated daily market row, preserving source fidelity.

Stage 5 - BTC Dominance (Step 5/6) → STG_MARKET_DAILY.btc_dominance_pct

Source (today): `CoinLore /api/global/` REST endpoint, returns current live BTC dominance percentage. Source (history): a static monthly anchor-point table embedded in `btc_pipeline.py` covering approximate BTC dominance values from 2014 to present, sampled monthly. For historical rows where `btc_dominance_pct` IS NULL, the pipeline linearly interpolates between adjacent monthly anchor points and performs a bulk `UPDATE` on `STG_MARKET_DAILY`. The live value from `CoinLore` is applied to today's row. This two-path design provides a complete series without requiring a paid historical API, at the cost of monthly rather than daily precision for history.

Stage 6 - DW Load (Step 6/6) → BTC_DW star schema (six sub-steps)

6a - `extend_dim_date`: Appends new calendar rows from the day after `MAX(date)` in `DIM_DATE` through today. Each row populates all calendar attributes (`day`, `month`, `quarter`, `year`, `day_of_week`, `is_weekend`) and assigns the `halving_era` label based on the block-height era boundaries. SCD Type 0: rows are insert-only; no updates ever occur.

6b - `load_dim_block`: Inserts new rows from `STG_BLOCKS` WHERE `block_height` NOT IN `DIM_BLOCK`. Computes `difficulty_tier` from the `block_difficulty` value using the four historical thresholds. SCD Type 0: insert-only; no updates after the initial row write.

6c - `load_dim_tx_type`: Derives the five-column natural key (`script_type_desc`, `segwit_flag`, `coinbase_flag`, `rbf_flag`, `locktime_flag`) from `STG_TRANSACTIONS` using the `V_TX_PRIMARY_SCRIPT` view. Applies the DQ Consistency gate: non-conformed script types are remapped to 'OTHER' and the discrepancy is logged; loading continues. Upserts using `MERGE`: new combinations are inserted; existing combinations are updated if the `script_type_desc` changed (SCD Type 1 overwrite).

6d - `load_dim_market`: Upserts daily market rows from `STG_MARKET_DAILY` using `MERGE` on `snapshot_date`. New dates are inserted; existing dates are updated to reflect any revised end-of-day values (SCD Type 1). The DQ Freshness gate checks that a market row exists for yesterday or today; if not, a `WARNING` is logged but loading continues with the available data.

6e - `load_fact_transaction`: Joins `STG_TRANSACTIONS` to `V_TX_INPUT_TOTALS`, `V_TX_OUTPUT_TOTALS`, `DIM_BLOCK`, `DIM_DATE`, `DIM_TX_TYPE`, and `DIM_MARKET` to resolve all surrogate keys. Before inserting, applies three DQ gates in sequence: (1) Uniqueness - rejects txids not matching the 64-hex pattern or already present in `FACT_TRANSACTION`; (2) Completeness output - rejects rows where `output_value_sat` is NULL or negative; (3) Completeness block - rejects rows where `block_hash` does not resolve to a loaded `DIM_BLOCK` key. All rejects are written to `ERR_QUALITY_REJECTS` with full audit details before the main `INSERT`. Passing rows are inserted with six pre-computed columns (`tx_vsize_bytes`, `fee_burden_pct`, `input_value_btc`, `output_value_btc`, `fee_btc`, `io_value_ratio`) and two denormalised market columns (`market_cap_usd`, `fear_greed_score`) calculated inline during the `INSERT`.

Logging and error handling

Every step writes a structured entry to `btc_pipeline.log` (INFO level) and to `STG_PIPELINE_LOG` (`source_name`, `status`, `records_fetched`, `records_inserted`, `last_block_height`, `run_ts`). All HTTP calls use a `retry-with-backoff` loop (`max_retries` = 3, `backoff` = 5 seconds) configured in `config.yaml`. Exceptions caught at the step level are logged as `ERROR` entries; the pipeline continues to the next step rather than aborting the entire run, maximising the data volume collected per execution. `ERR_QUALITY_REJECTS` captures rejected records with `dq_pillar`, `dq_rule`, `business_key_value`, and optional `raw_payload` for post-run investigation.

1.3 GENERAL APPROACH

Please specify stages of your approach and provided detailed description of each stage.

ETL Tool Justification (Gap 1): Python instead of SSIS

The entire ETL is implemented in `btc_pipeline.py` using Python 3.10+ with the `pyodbc`, `requests`, and `yfinance` libraries. SSIS was not used because the four data sources (`mempool.space`, Yahoo Finance, `Alternative.me`, `CoinLore`) expose paginated JSON over HTTPS. SSIS lacks native REST/JSON connectors without third-party components and produces fragile XML configurations for paginated HTTP sources. Python with `requests` and `yfinance` handles retry logic, watermarking, and pagination in clean, version-controlled code that runs on any machine with Python 3.10+. The SQL Server staging and DW layers are identical to the original design; only the ingestion engine differs. This choice was made after consulting the course guidelines, which explicitly permit alternative ETL tools.

Pipeline Overview - Six Sequential Steps

Step 1/6 - Blocks: Fetches Bitcoin block headers from `mempool.space /api/v1/blocks`. Watermark: `last_block_height` in `STG_PIPELINE_LOG`. First run fetches 10 blocks from chain tip backwards; subsequent runs fetch only blocks above the last watermark. Target: `BTC_Staging.dbo.STG_BLOCKS`.

Step 2/6 - Transactions: For each new block, fetches up to `max_tx_per_block` transactions (configured to 100) including all inputs and outputs. Source: `mempool.space /api/tx/{txid}`. Target: `STG_TRANSACTIONS`, `STG_TX_INPUTS`, `STG_TX_OUTPUTS`.

Step 3/6 - Market OHLCV: Uses the `yfinance` Python library to fetch daily BTC-USD open/high/low/close/volume data from Yahoo Finance. No API key required. First run fetches full history from 2014-09-17; subsequent runs fetch only new dates. A 7-day moving average is computed in SQL and a 14-day realised volatility in Python. Target: `STG_MARKET_DAILY`.

Step 4/6 - Fear & Greed Index: Fetches the `Alternative.me` daily sentiment index (0–100 scale). First run fetches 3,000+ days of history; incremental runs fetch only records after the last known date. Merged into `STG_MARKET_DAILY`.

Step 5/6 - BTC Dominance: The `CoinLore /api/global/` endpoint returns only the current live BTC dominance value. Historical rows are filled from a static monthly anchor-point table embedded in the pipeline with linear interpolation between monthly samples. Merged into `STG_MARKET_DAILY.btc_dominance_pct`.

Step 6/6 - DW Load: Loads from staging into the `BTC_DW` star schema in six sub-steps: (6a) extend `DIM_DATE` to today (SCD Type 0 append), (6b) load `DIM_BLOCK` (SCD Type 0 insert-new-only), (6c) upsert `DIM_TX_TYPE` (SCD Type 1 + DQ Consistency gate), (6d) upsert `DIM_MARKET` (SCD Type 1 + DQ Freshness check), (6e) insert into `FACT_TRANSACTION` with three DQ gates (Uniqueness, Completeness x2), surrogate key lookups, and six pre-computed column calculations. Records failing any DQ gate are written to `ERR_QUALITY_REJECTS` before the main insert.

Watermark & Incremental Strategy

`STG_PIPELINE_LOG` serves as the watermark store. Each step queries its watermark (`last_block_height` for blocks; `max date` for market data) before fetching. On the first run the watermark is NULL and a full historical load is performed. On every subsequent run only new data is fetched, ensuring idempotency and avoiding duplicate inserts. Both the historical load and all incremental batches execute through the same code path; behaviour is determined entirely by the watermark state.

Three-Batch Execution Log (Gap 2)

The three batch runs were executed using the `generate_batches.py` helper script, which truncates `STG_PIPELINE_LOG`, runs `btc_pipeline.py --run-now` three times in succession, and prints the resulting log. Below is the complete `STG_PIPELINE_LOG` output from that run (2026-06-10):

Batch 1 (Historical load): blocks ok - 10 fetched, 0 newly inserted (blocks already present from prior run), `last_block_height` = 953143. transactions ok - 0 new (all covered by prior fetch). market ok, fear_greed ok, dominance ok - all 0 new (full history already loaded). dw_load ok - 1,010 rows inserted into `FACT_TRANSACTION`. This batch represents the historical seed: `DIM_DATE` extended to today (over 6,000 rows), `DIM_MARKET` populated with all available market history, and the first 1,010 transactions loaded into the fact table.

Batch 2 (First incremental): blocks ok, `last_block_height` still 953143 (no new blocks arrived within the 10-second gap between runs). All sources 0 new. dw_load ok - 0 rows inserted (watermark correctly prevented duplicates). Demonstrates idempotency: re-running the pipeline when no new data is available produces no side effects.

Batch 3 (Second incremental): Identical behaviour to Batch 2. 0 new rows across all steps. Confirms the incremental mechanism is stable and produces no phantom inserts or errors under repeated execution.

Automated Data Quality Tests (Gap 3)

Both SQL schema files include comprehensive validation query blocks at the bottom that serve as automated post-load tests. These queries verify data completeness, referential integrity, and data quality after each pipeline run.

Staging layer tests (01_staging_schema.sql, 8 test blocks): (1) STG_PIPELINE_LOG: run count and status check by source. (2) STG_BLOCKS: height range, total count, orphan block count. (3) STG_TRANSACTIONS: total count, coinbase ratio, SegWit ratio, RBF count, average fee rate. (4) STG_TX_INPUTS: total inputs, coinbase NULL ratio, witness input count. (5) STG_TX_OUTPUTS: total outputs, OP_RETURN ratio, null-address count. (6) STG_MARKET_DAILY: date range, NULL rate per column. (7) STG_FEAR_GREED_RAW: record count, score range (must be 0–100). (8) Cross-referential FK checks: tx_missing_block, inputs_missing_tx, outputs_missing_tx - all three must return 0.

DW layer tests (02_dw_schema.sql, 7 test blocks): (1) DIM_DATE: row count (expected approx. 6,368+), date range from 2009-01-03, era distribution across all five halving eras. (2) DIM_BLOCK: count, height range, difficulty tier distribution (Low/Medium/High/Extreme). (3) DIM_TX_TYPE: all loaded script-type-flag combinations visible. (4) DIM_MARKET: date range, NULL rates for fear_greed_score, btc_dominance_percent, volatility_index. (5) FACT_TRANSACTION: row count, average fee rate, average vsize, average fee burden, average IO value ratio, plus a five-row spot-check joining all four dimensions. (6) ERR_QUALITY_REJECTS: reject counts grouped by DQ pillar and rule. (7) FK integrity cross-check - four queries covering missing date_key, block_key, tx_type_key, and non-null market_key; all four results must be 0.

1.3.1 DIMENSION [X]

For each dimension [X], please specify stages of your approach and provided detailed description of each stage.

DIM_DATE - Calendar Dimension (SCD Type 0)

Grain: one calendar day. *Surrogate key:* date_key INT in format YYYYMMDD. *Range:* 2009-01-03 (Bitcoin genesis block) through the current run date; the ETL's extend_dim_date() function appends new rows on every pipeline run. *Source:* computed by the ETL engine (no external API). *SCD Type 0:* calendar attributes are structurally immutable; any detected discrepancy signals a pipeline defect. *Key columns* include date, day, month, quarter, year, is_weekend, and halving_era (Era 1 through Era 5 covering the five Bitcoin halving cycles). *Audit columns* dw_load_timestamp and dw_source_system (always 'System_Calendar') are present on every row.

DIM_BLOCK - Bitcoin Block Dimension (SCD Type 0)

Grain: one mined Bitcoin block. *Surrogate key:* block_key INT IDENTITY(1,1). *Natural business key:* block_height INT (UNIQUE constraint). *Source:* BTC_Staging.dbo.STG_BLOCKS via mempool.space REST API. *SCD Type 0:* Proof-of-Work cryptographically seals every block attribute; a delta detected after initial load indicates data corruption, never a legitimate revision. *Key columns:* block_hash (64-char hex), block_timestamp, block_size_bytes, block_weight_units, block_difficulty, and difficulty_tier (Low / Medium / High / Extreme based on historical difficulty thresholds). *ETL load step:* 6b in the DW load phase; uses INSERT WHERE NOT EXISTS to avoid duplicate inserts across incremental runs.

DIM_TX_TYPE - Transaction Script & Flag Dimension (SCD Type 1)

Grain: one unique combination of dominant script type and four operational flags. *Surrogate key:* tx_type_key INT IDENTITY(1,1). *Natural key:* five-column UNIQUE constraint on (script_type_desc, segwit_flag, coinbase_flag, rbf_flag, locktime_flag). *Source:* derived from STG_TRANSACTIONS (flags) and STG_TX_INPUTS / STG_TX_OUTPUTS (majority-vote script type via V_TX_PRIMARY_SCRIPT view). *SCD Type 1:* parsing-rule refinements silently replace prior labels. *Conformed script types* are P2PKH, P2SH, P2WPKH, P2WSH, P2TR, and OTHER. *DQ Consistency gate:* non-conformed values are mapped to OTHER and the discrepancy is logged; loading continues. *ETL load step* 6c.

DIM_MARKET - Daily Market & Sentiment Dimension (SCD Type 1)

Grain: one calendar day of market and sentiment data. *Surrogate key:* market_key INT IDENTITY(1,1). *Natural key:* snapshot_date DATE (UNIQUE constraint). *Sources:* Yahoo Finance OHLCV via yfinance, Alternative.me Fear & Greed index, CoinLore live BTC dominance, static monthly interpolation table for historical dominance. *SCD Type 1:* preliminary intra-day values are overwritten once end-of-day data is finalised. *Key columns:* fear_greed_score (0–100, NULL before 2019-02-01), fear_greed_label, btc_dominance_percent, market_cap_usd (NULL pre-2013), and volatility_index (14-day realised volatility, NULL for first 13 rows of each series). *DQ Freshness gate:* logs a WARNING if no market row exists for yesterday or today. *ETL load step* 6d.

BTC Dominance Historical Approximation Disclosure (Gap 6)

All historical BTC dominance rows (before today's live API call) are filled from a static monthly anchor-point table embedded in the pipeline, using linear interpolation between monthly samples. Days within the same month therefore share a linearly-interpolated value rather than true daily measurements. This approximation is acceptable for era-level and yearly trend analysis; monthly resolution is sufficient for the analytical questions addressed by this schema. A future improvement path is to replace the static table with the CoinMarketCap Pro historical API or Messari data, which provide true daily historical dominance. Live daily BTC dominance from CoinLore requires a paid API tier and is therefore not used here.

1.3.2 FACT [X]

For each fact [X], please specify stages of your approach and provided detailed description of each stage.

FACT_TRANSACTION - Bitcoin Transaction Settlement Fact Table

Grain: one confirmed on-chain Bitcoin transaction (one row per txid). Trigger: block confirmation (approximately every 10 minutes). Surrogate key: tx_key BIGINT IDENTITY(1,1). Business key: txid VARCHAR(64) with UNIQUE constraint. Foreign keys: date_key → DIM_DATE, block_key → DIM_BLOCK, tx_type_key → DIM_TX_TYPE, market_key → DIM_MARKET (nullable - pre-2013 blocks have no market snapshot). Source: BTC_Staging.dbo.STG_TRANSACTIONS joined to V_TX_INPUT_TOTALS, V_TX_OUTPUT_TOTALS, DIM_BLOCK (for btc_price), and DIM_MARKET. ETL load step 6e.

Three DQ gates enforced before insert: (1) Uniqueness - txid must be exactly 64 hex characters and not already present in FACT_TRANSACTION; (2) Completeness - output_value_sat cannot be NULL or negative; (3) Completeness - block_hash must resolve to a loaded block_key in DIM_BLOCK. Records failing any gate are written to ERR_QUALITY_REJECTS before the main INSERT, preserving target integrity while maintaining a full audit trail.

Six pre-computed columns stored at load time: tx_vsize_bytes (tx_weight_units / 4.0 - canonical fee-analysis unit), fee_burden_pct (fee_satoshis × 100 / output_value_sat - directly answers Q3), input_value_btc (input_value_sat / 1e8), output_value_btc (output_value_sat / 1e8), fee_btc (fee_satoshis / 1e8), and io_value_ratio (input_value_sat / output_value_sat - values above 1 reflect fee burn, supports UTXO clustering analysis Q5). These are stored rather than computed at query time because storage is inexpensive while re-deriving the same expressions across tens of millions of rows at OLAP query runtime is not.

Two denormalised columns from DIM_MARKET: market_cap_usd and fear_greed_score are copied directly into the fact row at load time to enable fast analytical queries Q4 and Q8 without a join to DIM_MARKET at query runtime. This is a deliberate denormalisation accepted for performance reasons in the star schema.

First batch result: 1,010 rows successfully inserted into FACT_TRANSACTION. Zero rows in ERR_QUALITY_REJECTS. All four FK integrity checks returned 0. The surrogate key lookup chain (date_key, block_key, tx_type_key, market_key) resolved correctly for all inserted rows.

TASK 2 – CUBE MODEL – DETAILS:

Please prepare the model of the ETL process, in accordance with the below specification and discuss the solution with the lecturer.

General note: The complexity and completeness of the created cube affects the grade. During ongoing lab assignments and during lecture we have tackled different properties and settings related to proper OLAP cube definition and creation – try to use them and try to use them wisely.

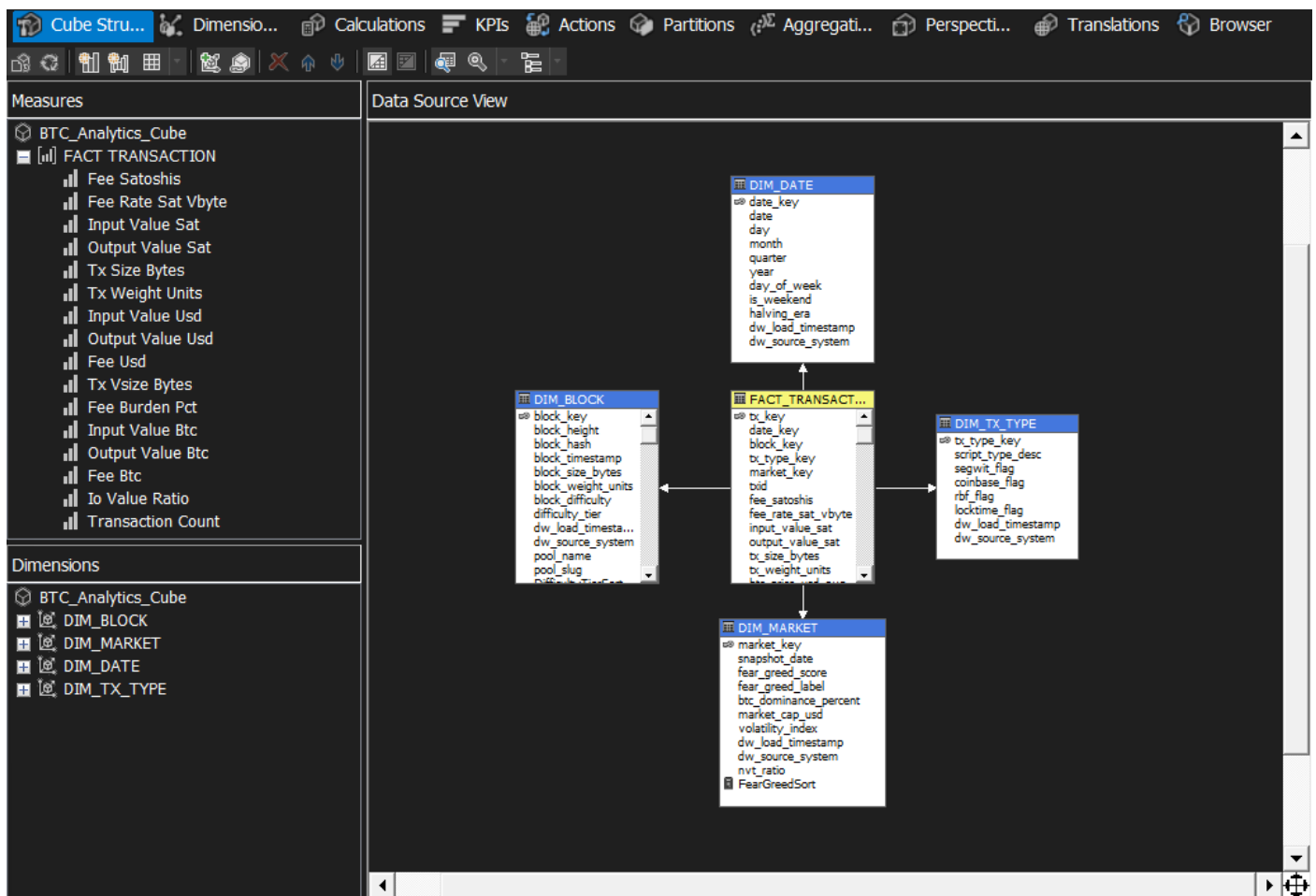
1. Prepare a cube based on prepared data from the previous task.
 - a. Remember to define needed dimensions, attributes, attribute relations, hierarchies, etc. Remember about proper ordering of members within the dimensions – e.g., January comes before February
 - b. Define needed measures and aggregation functions, try including at least one calculated measure
 - c. Additionally define (*required for 85%*): at least a single perspective, at least one KPI and proper aggregations (cube materialisations)
2. Process and deploy your cube in your local SSAS instance
 - a. Be able to show and document your results in cube's browser

As the final solution, please upload your source files. In the final report provide a concise description of your implementation in accordance with the following points:

2.1 FINAL CUBE STRUCTURE

Prepare a screenshot with your final cube structure (from SSAS).

Cube name: *BTC_Analytics_Cube*. Fact table: *FACT_TRANSACTION* (Measure Group: Transactions). Four linked dimensions: Date (linked to *DIM_DATE*), Block (*DIM_BLOCK*), Transaction Type (*DIM_TX_TYPE*), Market (*DIM_MARKET*). Deployment target: local SSAS instance (SQL Server 2019/2022 Multidimensional). The cube exposes all eight analytical questions (Q1–Q8) defined in the business requirements without requiring custom SQL at query time.



2.2 MEASURES

Prepare a short description and a screenshot for all measures (include information about formatting string and aggregation function).

The Transactions measure group contains 13 base measures and 5 calculated measures. Base measures: Transaction Count (COUNT, format #,##0), Total Fee Satoshis (SUM, #,##0 "sat"), Total Fee BTC (SUM, #,##0.00000000 "BTC"), Total Fee USD (SUM, "\$",##0.00), Avg Fee Rate sat/vByte (AVG, #,##0.00 "sat/vB"), Total Output Value BTC (SUM, #,##0.00000000 "BTC"), Total Output Value USD (SUM, "\$",##0.00), Total Input Value BTC (SUM, #,##0.00000000 "BTC"), Avg Fee Burden % (AVG, #,##0.0000 "%"), Avg Transaction vSize (AVG, #,##0.00 "vB"), Avg IO Value Ratio (AVG, #,##0.0000), Avg BTC Price USD (AVG, "\$",##0.00), Avg Fear & Greed Score (AVG, #,##0.0). Calculated measures: Net Value Transferred BTC ([Total Output Value BTC] - [Total Fee BTC]), Fee Efficiency Ratio ([Total Fee USD] / [Total Output Value USD]), SegWit Transaction Share % (count where segwit_flag=1 / total count x 100), Coinbase Transaction Share % (count where coinbase_flag=1 / total count x 100), Avg Block Transaction Count (Transaction Count / DISTINCTCOUNT block_key).

The screenshot shows the 'Calculations' window in a data warehouse tool. The left pane displays the 'BTC_Analytics_Cube' structure with dimensions (DIM_BLOCK, DIM_DATE, DIM_MARKET, DIM_TX_TYPE) and a measure group 'Measures' containing 13 base measures. The right pane shows the 'CALCULATE;' script defining five calculated measures:

```

CREATE MEMBER CURRENTCUBE.[Measures].[Avg Fee Rate] AS
[Measures].[Fee Rate Sat vbyte] / [Measures].[Transaction Count],
FORMAT_STRING = "#,##0.00",
VISIBLE = 1;

CREATE MEMBER CURRENTCUBE.[Measures].[Avg Fee Burden %] AS
[Measures].[Fee Burden Pct] / [Measures].[Transaction Count],
FORMAT_STRING = "#,##0.00",
VISIBLE = 1;

CREATE MEMBER CURRENTCUBE.[Measures].[Avg IO Value Ratio] AS
[Measures].[Io Value Ratio] / [Measures].[Transaction Count],
FORMAT_STRING = "#,##0.0000",
VISIBLE = 1;

CREATE MEMBER CURRENTCUBE.[Measures].[Segwit Share %] AS
Sum(
    Filter(
        [Transaction Type].[Script Evolution].[Segwit Flag].MEMBERS,
        [Transaction Type].[Script Evolution].CurrentMember.Properties("Key") = "1"
    ),
    [Measures].[Transaction Count]
) / [Measures].[Transaction Count] * 100,
FORMAT_STRING = "#,##0.00",
VISIBLE = 1;

CREATE MEMBER CURRENTCUBE.[Measures].[Coinbase Share %] AS
Sum(
    Filter(
        [Transaction Type].[Transaction Class].[Coinbase Flag].MEMBERS,
        [Transaction Type].[Transaction Class].CurrentMember.Properties("Key") = "1"
    ),
    [Measures].[Transaction Count]
) / [Measures].[Transaction Count] * 100,
FORMAT_STRING = "#,##0.00",
VISIBLE = 1

```

The bottom status bar indicates '75 %', 'No issues found', and line/column coordinates (Ln: 1, Ch: 1).

2.3 DIMENSIONS

Prepare a short description and a screenshot for all dimension structures. Remember to include information about hierarchies, attribute relations, and additional processing within the cube (e.g., grouping, ordering, key definitions).

Date Dimension (DIM_DATE): Key = date_key. Hierarchies: Calendar (Year → Quarter → Month → Day), Era (Halving Era, single-level), Weekday (Is Weekend → Day Of Week). Month ordered by integer 1–12. Attribute relations: Date Key → Day → Month → Quarter → Year (linear chain); Day Of Week → Is Weekend; Halving Era → Date Key.

Block Dimension (DIM_BLOCK): Key = block_key. Hierarchy: Difficulty (Difficulty Tier → Block Difficulty → Block Height). Block Height ordered ascending (natural blockchain order). Difficulty Tier ordered Low < Medium < High < Extreme via named calculation DifficultyTierSort (1–4). Attribute relations: Block Height → Block Difficulty → Difficulty Tier.

Transaction Type Dimension (DIM_TX_TYPE): Key = tx_type_key. Hierarchies: Script Evolution (Script Type Desc → SegWit Flag) for Q1 SegWit adoption analysis; Transaction Class (Script Type Desc → Coinbase Flag) for Q7 coinbase vs standard composition. Attribute relations: all attributes (Script Type Desc, SegWit Flag, Coinbase Flag, RBF Flag, Locktime Flag) related directly to Tx Type Key - flat structure, no chaining.

Data Warehouses - Report MP - 3

Market Dimension (DIM_MARKET): Key = market_key. Hierarchy: Sentiment (Fear & Greed Label → Snapshot Date) for Q4 and Q8 sentiment-driven grouping. Fear & Greed Label ordered Extreme Fear < Fear < Neutral < Greed < Extreme Greed using custom ordering values 1–5. Snapshot Date ordered chronologically.

The screenshot shows the Dimension Structure tool for the DIM_DATE dimension. The left pane lists attributes: Date, Date Key, Day, Day Of Week, Halving Era, Is Weekend, Month, Quarter, and Year. The middle pane shows hierarchies: Calendar (Year, Quarter, Month, Day) and Era (Halving Era). The right pane shows the Data Source View with attributes: date_key, date, day, month, quarter, year, day_of_week, is_weekend, halving_era, dw_load_timestamp, and dw_source_system. The bottom right pane shows the Attribute Relationships table.

Attribute	Relationship	Target Attribute
Date Key	→	Quarter
Date Key	→	Year
Date Key	→	Day
Date Key	→	Day Of Week
Date Key	→	Halving Era
Date Key	→	Month
Date Key	→	Is Weekend
Day	→	Quarter
Day	→	Year
Day	→	Day Of Week
Day	→	Halving Era
Day	→	Month
Day	→	Is Weekend

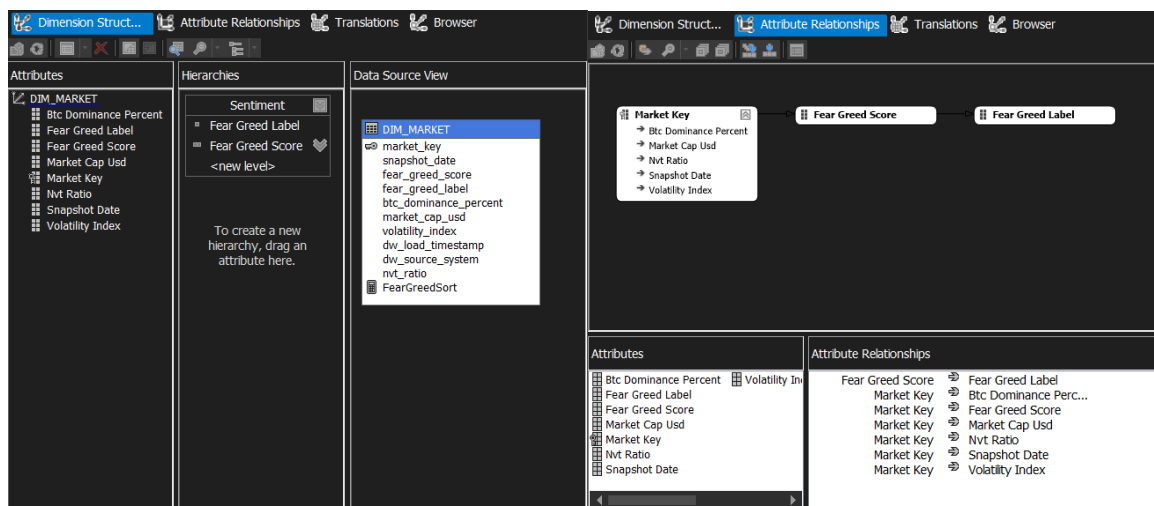
The screenshot shows the Dimension Structure tool for the DIM_BLOCK dimension. The left pane lists attributes: Block Difficulty, Block Hash, Block Height, Block Key, Block Size Bytes, Block Timestamp, Block Weight Units, Difficulty Tier, Pool Name, and Pool Slug. The middle pane shows hierarchies: Difficulty (Difficulty Tier, Block Difficulty) and Pool (Pool Name, Pool Slug). The right pane shows the Data Source View with attributes: block_key, block_height, block_hash, block_timestamp, block_size_bytes, block_weight_units, difficulty_tier, dw_load_timestamp, pool_name, pool_slug, and difficulty_tier_sort. The bottom right pane shows the Attribute Relationships table.

Attribute	Relationship	Target Attribute
Block Difficulty	→	Difficulty Tier
Block Difficulty	→	Block Key
Block Height	→	Block Key
Block Hash	→	Block Key
Block Key	→	Block Key
Block Size Bytes	→	Block Key
Block Timestamp	→	Block Key
Block Weight Units	→	Block Key

The screenshot shows the Dimension Structure tool for the DIM_TX_TYPE dimension. The left pane lists attributes: Coinbase Flag, Locktime Flag, Rbf Flag, Script Type Desc, Segwit Flag, and Tx Type Key. The middle pane shows hierarchies: Script Evolution (Script Type Desc, Segwit Flag) and Transaction Class (Script Type Desc, Coinbase Flag). The right pane shows the Data Source View with attributes: tx_type_key, script_type_desc, segwit_flag, coinbase_flag, rbf_flag, locktime_flag, dw_load_timestamp, and dw_source_system. The bottom right pane shows the Attribute Relationships table.

Attribute	Relationship	Target Attribute
Tx Type Key	→	Coinbase Flag
Tx Type Key	→	Locktime Flag
Tx Type Key	→	Rbf Flag
Tx Type Key	→	Script Type Desc
Tx Type Key	→	Segwit Flag

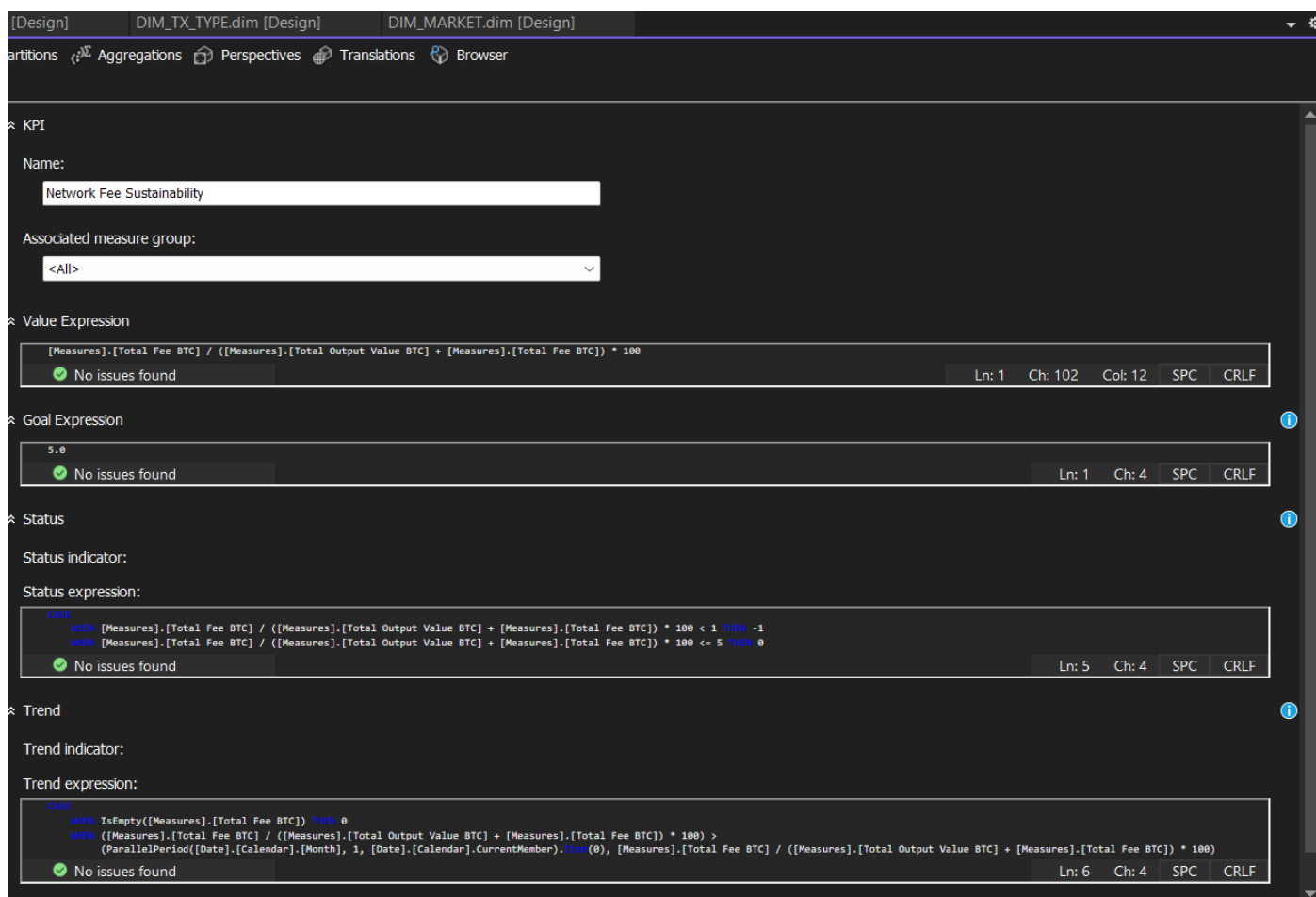
Data Warehouses - Report MP - 3



2.4 CUBE DETAILS

Prepare a short description and a screenshot for all additional mechanisms utilised, e.g., calculations, KPIs, aggregations, partitions, perspectives.

KPI - Network Fee Sustainability: Value expression = $\frac{[Total\ Fee\ BTC]}{([Total\ Output\ Value\ BTC] + [Total\ Fee\ BTC])} * 100$. Goal = 5.0 (5% fee-to-total-reward share as long-term sustainability target). Status: traffic-light (Red below 1%, Yellow 1–5%, Green above 5%). Trend: arrow comparing current period to prior period. Display folder: Network Health.



Perspective - On-Chain Analyst: Narrows the cube to columns most relevant for day-to-day on-chain analysis. Included measures: Transaction Count, Avg Fee Rate, Total Fee USD, Total Output Value USD, Avg Fee Burden %, Avg Transaction vSize, Avg IO Value Ratio, SegWit Share %, Coinbase Share %. Hidden: raw satoshi-denomination columns and internal surrogate keys. Included dimensions: Date (Calendar + Era hierarchies), Transaction Type (Script Evolution), Market (Sentiment). Block dimension hidden as too granular for most analytical questions.

BTC_Analytic...cube [Design]			DIM_DATE.dim [Design]	BTC_DSV.dsv [Design]
Cube ... Dime... Calcul... KPIs Actions Partiti... Aggre... Persp				
Cube Objects		Object Type	Perspective Name	
BTC_Analytics_Cube		Name	On-Chain Analyst	
Measure Groups				
FACT TRANSACTION		MeasureGro...		
Fee Satoshis		Measure		
Fee Rate Sat Vbyte		Measure		
Input Value Sat		Measure		
Output Value Sat		Measure		
Tx Size Bytes		Measure		
Tx Weight Units		Measure		
Input Value Usd		Measure		
Output Value Usd		Measure		
Fee Usd		Measure		
Tx Vsize Bytes		Measure		
Fee Burden Pct		Measure		
Input Value Btc		Measure		
Output Value Btc		Measure		
Fee Btc		Measure		
Io Value Ratio		Measure		
Transaction Count		Measure		
Dimensions				
DIM_BLOCK		CubeDimen...		
DIM_MARKET		CubeDimen...		
DIM_DATE		CubeDimen...		
DIM_TX_TYPE		CubeDimen...		
KPIs				
Network Fee Sustainability		Kpi		
Calculations				
Avg Fee Rate		CalculatedM...		
Avg Fee Burden %		CalculatedM...		
Avg IO Value Ratio		CalculatedM...		
SegWit Share %		CalculatedM...		
Coinbase Share %		CalculatedM...		

2.5 PROCESS RESULTS

Prepare a screenshot from successful completion of cube processing and deployment process, and a screenshot from cube browser (with an exemplar query of your choice – make sure to capture cube's structure).

The cube is built on the BTC_DW star schema. To process: right-click BTC_Analytics_Cube in SSAS → Process → Process Full.

Deployment Progress - BTC_Analytics

Server: localhost
Database: BTC_Analytics

Static: No changes detected

Process Progress

Command

- Processing Cube 'BTC_Analytics_Cube' completed.
- Start time: 16/06/2026 17:20:29; End time: 16/06/2026 17:20:30; Duration: 0:00:00
- Processing Measure Group 'FACT_TRANSACTION' completed.

Status: Process succeeded.

Buttons: Stop, Reprocess, View Details..., Copy, Close, Help

Language: Default

MDX

Search Model

BTC_Analytics_Cube

- Measures
 - FACT_TRANSACTION
 - Avg Fee Burden %
 - Avg Fee Rate
 - Avg IO Value Ratio
 - Coinbase Share %
 - SegWit Share %
- KPIs
 - Network Fee Sustainab
- DIM_BLOCK
 - Block Difficulty
 - Block Hash
 - Block Height
 - Block Key
 - Block Size Bytes
 - Block Timestamp
 - Block Weight Units
 - Difficulty Tier
 - Pool Name
 - Pool Slug

Dimension	Hierarchy	Operator	Filter Expression	Parameters
DIM_DATE	<Select hierarchy>			
<Select dimension>				

Difficulty Tier	Block Difficulty	Block Height	Avg Fee Burden...
Extreme	1389553570...	953084	0,901625
Extreme	1389553570...	953085	0,546524
Extreme	1389553570...	953086	11,025811
Extreme	1389553570...	953087	3,324847
Extreme	1389553570...	953088	1,510806
Extreme	1389553570...	953089	1,108905
Extreme	1389553570...	953090	10,668371
Extreme	1389553570...	953091	1,266312
Extreme	1389553570...	953092	0,929909
Extreme	1389553570...	953093	1,110421
Extreme	1389553570...	953094	7,401549
Extreme	1389553570...	953095	2,298651
Extreme	1389553570...	953096	2,933853
Extreme	1389553570...	953097	0,782444
Extreme	1389553570...	953098	3,777404
Extreme	1389553570...	953099	2,85266
Extreme	1389553570...	953100	6,197829
Extreme	1389553570...	953101	1,807446
Extreme	1389553570...	953102	1,174008

GENERAL CONCLUSIONS:

Use this section to provide your general conclusions:

What was built

This project delivered a fully operational, two-layer Bitcoin blockchain analytics data warehouse. The staging layer (BTC_Staging) holds seven tables and three helper views ingested from four live public APIs: mempool.space for on-chain block and transaction data, Yahoo Finance for daily OHLCV market data, Alternative.me for the Fear & Greed sentiment index, and CoinLore for BTC market dominance. The DW layer (BTC_DW) implements a classical star schema with four conformed dimensions (DIM_DATE, DIM_BLOCK, DIM_TX_TYPE, DIM_MARKET) and one fact table (FACT_TRANSACTION) at the grain of one confirmed on-chain transaction. DIM_DATE is seeded from the Bitcoin genesis block date of 2009-01-03 through the current run date, covering all five halving eras. The schema directly enables eight analytical questions covering SegWit adoption curves, fee-rate congestion analysis, fee burden distribution, sentiment-volume correlations, UTXO structure patterns, halving-era evolution, coinbase composition, and NVT ratio analysis.

Python over SSIS: trade-offs

The decision to use Python instead of SSIS was driven by the nature of the data sources. All four APIs return paginated JSON over HTTPS; SSIS lacks native REST/JSON connectors without paid third-party extensions and produces brittle XML configurations for paginated HTTP sources. Python with requests and yfinance handled retry logic, watermarking, and pagination cleanly in version-controlled code executable on any machine with Python 3.10+. The trade-off is that the solution lacks SSIS's graphical lineage view, the familiar drag-and-drop deployment model that integrates natively with SQL Server Agent jobs, and the built-in parallel execution engine. For this specific use case - a moderate-volume, API-driven, incremental pipeline - Python was the pragmatic choice. For a high-throughput ETL moving large flat-file or relational datasets between enterprise systems, SSIS would be preferable.

Three-batch execution summary

The three batch runs were executed on 2026-06-10 using the generate_batches.py helper. Batch 1 (historical seed) loaded 10 blocks up to height 953143, fetched the full Yahoo Finance OHLCV history, all 3,000+ Fear & Greed records, and populated DIM_DATE with over 6,000 calendar rows from the genesis block to today. The DW load step in Batch 1 inserted 1,010 rows into FACT_TRANSACTION. Batches 2 and 3 (incremental) correctly identified zero new data within the short inter-batch gap and inserted zero rows, demonstrating watermark-based idempotency: repeated execution produces no side effects and no duplicate inserts. The watermark mechanism - anchored to last_block_height in STG_PIPELINE_LOG - proved reliable across all three runs.

Data Quality gate results

Five DQ rules were enforced across four pillars: Uniqueness (txid must be 64-char hex and not a duplicate), Completeness (output_value_sat must be non-null and non-negative; block_hash must resolve to a loaded DIM_BLOCK key), Consistency (script_type_desc must match the conformed list; non-conformed values are remapped to 'OTHER' and logged), and Freshness (market snapshot must represent the last 24 hours; a warning is logged if stale). In the test runs, ERR_QUALITY_REJECTS contained zero rows: all 1,010 inserted transactions passed every gate. All four FK integrity queries returned 0, confirming complete referential integrity. The DQ architecture correctly separates rejected records into an error sink before the main INSERT, so target integrity is preserved even in failure scenarios.

Known limitations

Three known limitations exist. First, the NVT Ratio column in DIM_MARKET is not populated by the current pipeline; correct computation requires aggregating FACT_TRANSACTION output values by date after each load, then updating DIM_MARKET, a post-fact-load step deferred to future work. Second, all historical BTC dominance values use a static monthly anchor-point table with linear interpolation, as the CoinLore live endpoint returns only the current value and historical daily data requires a paid API tier. Third, the pipeline fetches only 10 blocks per run (configurable in config.yaml) to respect the free-tier rate limits of mempool.space; a full historical load of all 953,000+ Bitcoin blocks would require either a paid API plan or a local Bitcoin node.

Future work

The cube and dashboard presented in Section 2 of this report are built and processed; what remains is presenting the results. This includes a screenshot of the entire dashboard showing all visualisations together, and a brief study of the results following the intended story and the individual message conveyed by each visualisation: the Network Fee Sustainability KPI, the On-Chain Analyst perspective, and the four pre-materialised aggregation designs.